# Old vs new cost function

Electric Board Map

Kathan Prajapati · 6 November 2026

**The five-term cost leaves 23% fewer forced routes in crowded trees and draws 19% faster there. Board size and free space stay within 1% of the old nine-term cost.**

| Same inputs, same settings | Old cost (9 terms) | New cost (5 terms) | Change |
|---|---|---|---|
| Forced routes, 7 crowded forests | 73 | 56 | −23% |
| Problems found by the verifier, 7 crowded forests | 92 | 60 | −35% |
| Grid points per vertex, 10 test forests (one board each) | 4.50 | 4.53 | +0.5% |
| Grid points per vertex, 25 000 vertices on 63 boards | 4.66 | 4.64 | −0.4% |
| Free grid points on the board, 10 test forests | 56.3% | 56.5% | +0.2 points |
| Time, 10 test forests | 4.3 s | 4.2 s | −2% |
| Time, 25 000 vertices | 19.8 s | 16.0 s | −19% |
| Time, 7 crowded forests | 87.3 s | 70.3 s | −19% |

On the ten test forests neither cost leaves a conflict. On those boards the new cost is not uniformly smaller: six boards grow by 1–4%, one shrinks by 8%, and three are equal.

## The two cost functions

The new cost keeps the four terms that describe the trace itself and merges the two look-ahead terms into one. Every legal candidate trace $\pi$ for the child $v$ of $u$, ending at $e$ with $b(\pi)$ bends, gets a cost; the cheapest wins.

**Old cost, 9 terms.**

$$\begin{aligned} C_{\text{old}}(\pi) = {} & \ell(\pi) - \sigma\,\text{bus}(\pi) + 0.7\,b(\pi) + \lambda\,\Delta H(\pi) + 60\,\big[\kappa(v) - \varphi'(e)\big]^{+} + \mu\,\big[W(v) - \rho(e)\big]^{+} \\ & - \mathbf{1}\big[d_1 = \delta_{\text{in}}(u)\big] - \omega\big(\|e\|_\infty - \|p(u)\|_\infty\big) + 400\,\chi(\pi) \end{aligned}$$

**New cost, 5 terms.**

$$C_{\text{new}}(\pi) = \ell(\pi) + 0.7\,b(\pi) + \lambda\,\Delta H(\pi) - \sigma\,\text{bus}(\pi) + 60\,\big[\kappa(v) - \min\big(\varphi'(e), \Lambda'(e)\big)\big]^{+}$$

Here $\ell$ is the trace length, $\Delta H$ the growth of the board's half perimeter, bus the number of steps parallel to a neighbouring trace, $\kappa(v)$ the number of children of $v$, $\varphi'(e)$ the free directions at the new pad and $\Lambda'(e)$ its free landing points two steps away.

## Conflicts

On crowded forests the new cost leaves 56 forced routes where the old one left 73, and the verifier finds 60 problems instead of 92. The gain is not uniform: four forests improve, three get slightly worse (Figure 1).

On the ten test forests and on 25 000 vertices in 63 boards, neither cost forces a single route. In the browser run of $10^6$ vertices on 2500 boards, the new cost forced no route either; the verifier reported 54 problems, 5.4 per 100 000 vertices.

| Term | Old | New |
|---|---|---|
| Length $\ell$, bends $0.7\,b$, compactness $\lambda\,\Delta H$, bus $-\sigma$ bus | kept | kept, unchanged |
| Exit look-ahead | 60 per child beyond the free directions $\varphi'$ | 60 per child beyond $\min(\varphi', \Lambda')$: a free direction *and* a free landing point |
| Room look-ahead $\mu\,[W-\rho]^{+}$ ($\beta$, $\mu$, $R_{\max}$) | yes | removed; its job is done by $\Lambda'$ |
| Line continuation (bonus 1) | yes | removed |
| Radial growth $\omega$ (one retry variant) | yes | removed, with the radial and roomy retries |
| Conflicts $400\,\chi$ in forced mode | yes | replaced by ranking: fewest conflicts first, then cost |
| Parameters the user can set | $\lambda$, $\sigma$ | $\lambda$, $\sigma$ |
| Fixed constants | 0.7, 60, 1, 400, $\beta$, $\mu$, $R_{\max}$, $\omega$ | 0.7, 60 |

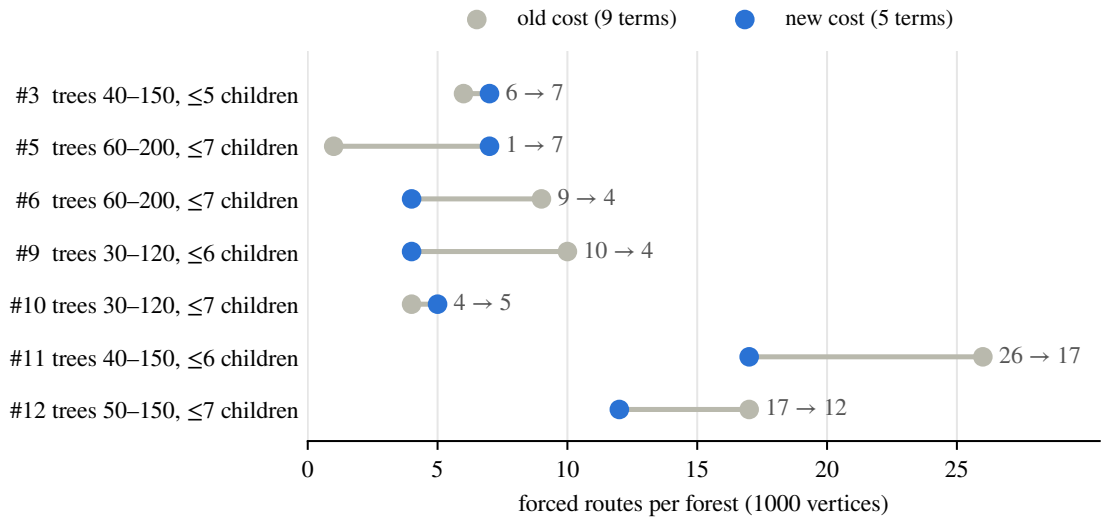



**Figure 1:** Forced routes in seven crowded forests (1000 vertices each). Total 73 → 56; fewer in 4 of 7 forests.

## Space left on one board

The two costs leave the same amount of space: about 56% of a board's grid points hold neither a pad nor a trace with either cost (56.3% old, 56.5% new). Per board the size moves both ways, by up to 8% (Figure 2).

Most of the free space is forced by the pad clearance: a pad keeps the 8 points around it free of other pads, which is why no board can go below about 4 grid points per vertex. Both costs stay about 13% above that bound on average. On 25 000 vertices in 63 boards the new cost is slightly denser, 4.64 against 4.66 grid points per vertex.

## Time to draw

The new cost is as fast as the old one on easy boards and about a fifth faster where routing is hard. The likely reasons are the cheaper look-ahead (no window count around every end point) and fewer retries, since retries run only while routes conflict.

In the browser with two worker threads, the new cost draws $10^6$ vertices on 2500 boards in 4.8 minutes, about 3500 vertices per second. Branch and bound still finds exactly the same traces as a full search, 3.9 times faster (0.49 s against 1.91 s on ten random trees of 32–140 vertices).

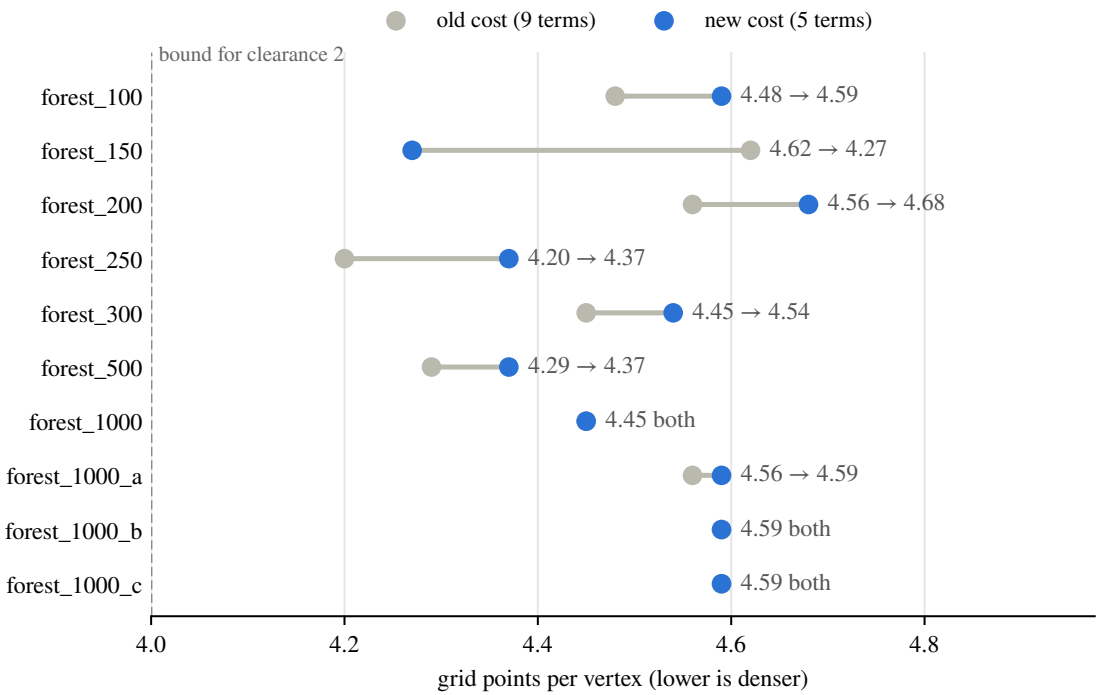



**Figure 2:** Board size on the ten test forests. All ten boards together: 4.50 grid points per vertex old, 4.53 new.

| Input | Old | New |
|---|---|---|
| One board, 100–300 vertices (5 forests) | 0.98 s | 0.95 s |
| One board, 500–1000 vertices (5 forests) | 3.35 s | 3.29 s |
| 25 000 vertices, 63 boards, one thread | 19.8 s | 16.0 s |
| 7 crowded forests, 1000 vertices each | 87.3 s | 70.3 s |

## How it was measured

Both versions of `large.html` (repository version and the simplified one) ran the same routing engine headless in Node.js, one thread, two cores, default settings: $L = 6$, clearance 2, $\lambda = 1.5$, $\sigma = 0.5$, aspect 1.6.

- **Test forests**: the ten files `forest_100` … `forest_1000_c` from the repository, each on one board, squeeze budget 800 ms.
- **25 000 vertices**: `forest_25000` split into 63 boards of 396–397 vertices, squeeze budget 150 ms per board.
- **Crowded forests**: seven generated forests of 1000 vertices, half in large trees of 30–200 vertices with up to 5–7 children per vertex (seeds 3, 5, 6, 9, 10, 11, 12). These are harder than the target class and are where conflicts appear.
- **Grid points per vertex** = (board width $+1$) $\times$ (board height $+1$) in grid steps, divided by the number of vertices. The bound for clearance 2 is about 4.
- **Free grid points** = share of the board's grid points that hold neither a pad nor a trace point.
- **Forced routes** = traces the router had to draw with conflicts. **Verifier problems** = what the independent check reports (overlaps, traces through pads).

Caveats: retries and the squeeze stop after a time budget, so times and conflict counts vary a little from run to run ($\pm 2$ conflicts on the crowded set). Board sizes repeated exactly over three runs. Both versions still show one verifier problem on `forest_25000` that is not a forced route (a trace passing through a pad), which the old version had too.